

공세민

Se Min Kong

소프트웨어 개발자

안녕하세요. 공세민입니다.

숭실대학교에서 소프트웨어를 전공했습니다.

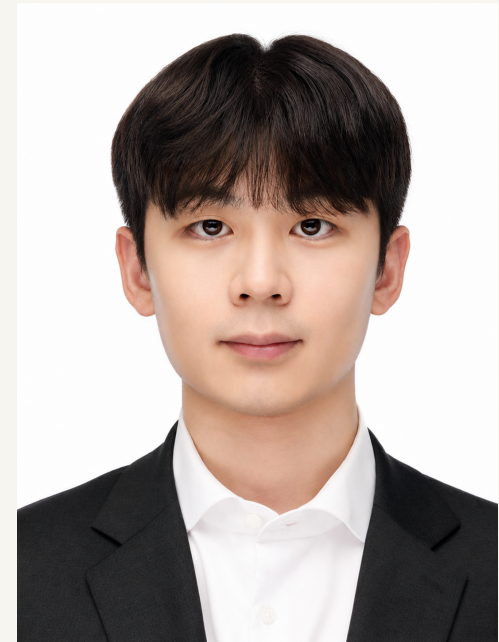
현재 SSAFY Robotics Track에서 공부하고 있습니다.

Computer Vision과 로봇 제어, Physical AI에 관심이 있습니다.

Python과 C++를 사용합니다.

semin1224@gmail.com

github.com/SeMinKong



현재

SSAFY Robotics Track
2026.01 - 현재

학력

숭실대학교
소프트웨어학부
2020.03 - 2026.02

관심 분야

Computer Vision
Robotics / Physical AI

거주지

Suwon,
Republic of Korea

학력과 관심 분야

학력·교육

숭실대학교 소프트웨어학부

2020.03 - 2026.02

소프트웨어 공학사

인공지능·빅데이터 전공

빅데이터 융합전공

SSAFY Robotics Track

2026.01 - 현재

삼성청년SW·AI아카데미 교육생

로보틱스, Computer Vision, ROS 2와

하드웨어·소프트웨어 통합을 공부합니다.

병역 2021.05 - 2022.11

육군 전술 통신 장비 운용·정비 / 병장 전역

어학 OPIc IH / 2027.10까지 유효

관심 분야

Computer Vision & Robotics

지금은 Computer Vision과 ROS 2, 로봇 제어를 중심으로 배우고 있습니다. 소프트웨어가 실제 장치와 만나는 로보틱스와 Physical AI에 관심이 있습니다.

Simulation

Isaac Sim과 Isaac Lab은 현재 학습 중인 도구입니다. 센서 오차, 통신 지연, 접촉과 마찰처럼 시뮬레이션과 실제 환경 사이에서 달라지는 조건에 관심을 두고 있습니다.

Local AI & Software

Ollama와 llama.cpp 등 로컬 AI 도구를 살펴보고 있습니다. Python과 C++를 사용하며, Ubuntu, Git, Docker와 같은 개발 환경도 함께 다룹니다.

프로젝트

쪽	프로젝트	역할	주요 기술
4 - 7	THING	6인 팀 / 구동·기구 통합	DYNAMIXEL · U2D2 · 제어 스크립트
8 - 10	AQIS	2인 팀 / 팀장·서버·장비 통합	FastAPI · WebSocket · ROS 2 · Dobot
11 - 13	Briefit	6인 팀 / AI 담당	Python · 비동기 수집 · KoBART
14 - 16	Brain MRI	개인 / 전처리·학습·통합 추론	YOLO11 · PyTorch · OpenCV
17 - 18	Alkkagi.io	개인 / 클라이언트·서버·물리	TypeScript · React · Socket.IO
19	Prompt Generator	개인 / 대화 서버·상태 관리	FastAPI · WebSocket · LangChain

THING · 텐던 로봇 핸드

기간

2026.07 - 08

팀·역할

6인 팀

모터 제어·기구 통합

사용 도구

DYNAMIXEL / U2D2

모터 점검·제어 스크립트

팀 기술

ROS 2 / MediaPipe

OpenCV / Jetson

Raspberry Pi 5



모터 고정부·스풀·텐던을 연결한 통합 조립 상태

프로젝트 개요

카메라에서 읽은 손동작을 7개 논리축 명령으로 바꾸고 텐던 로봇손을 구동합니다. 엄지는 3축, 나머지 네 손가락은 각각 1축으로 다룹니다. 팀 시스템은 21개 손 landmark를 인식하고, 명령 중재와 guard를 거쳐 모터를 구동하도록 구성했습니다.

담당 작업

모터 통신 환경과 U2D2 연결, 7개 모터 점검·제어 스크립트, 전완부 아크릴 고정부 제작과 스푼·텐던 통합을 맡았습니다. 통신 확인, 개별 제어, 실제 조립을 각각 점검할 수 있도록 작업을 나눴습니다.

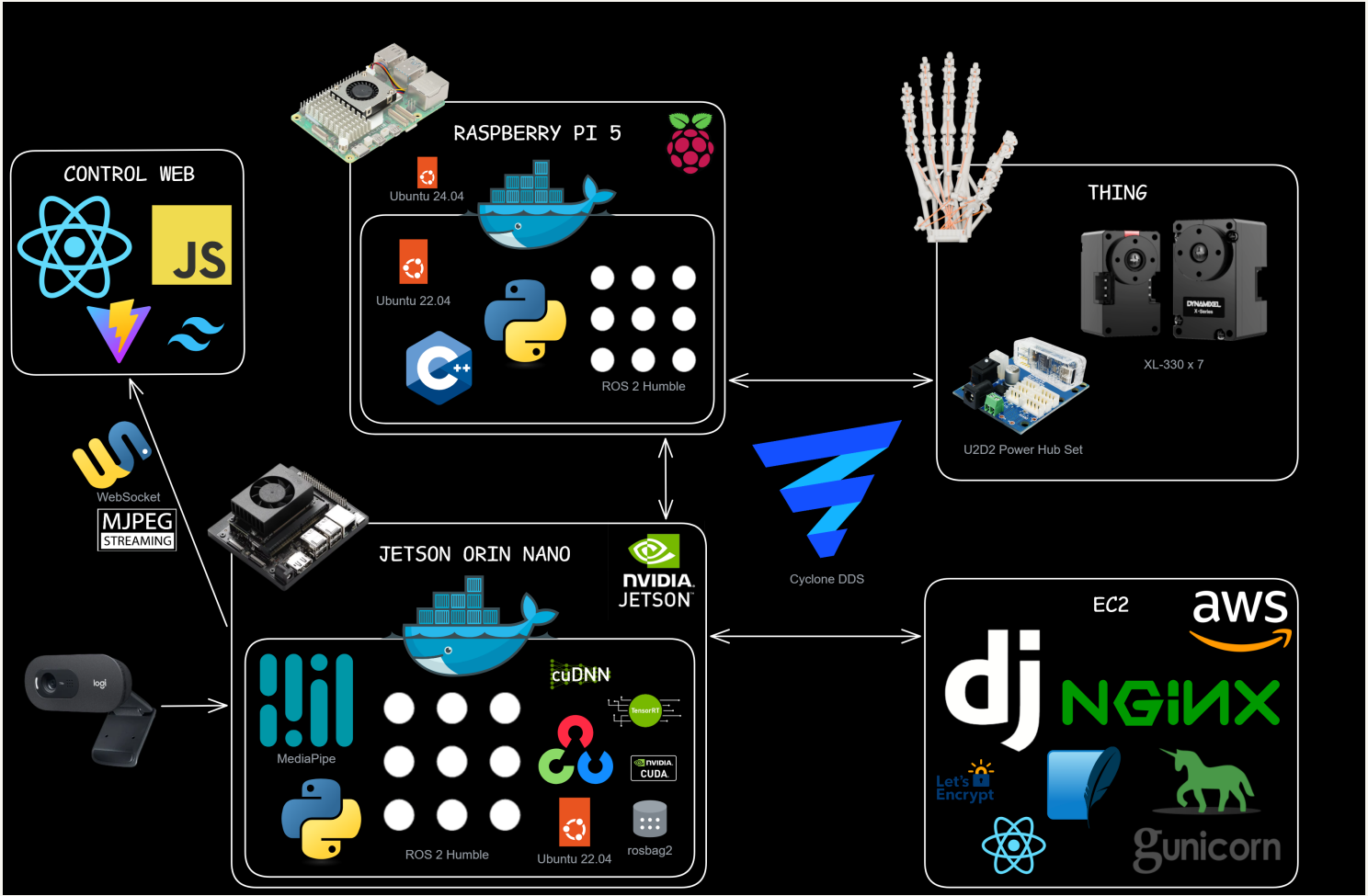
협업 범위

손동작 인식, ROS 2 명령 중재·guard, 관제와 데이터 기록은 팀 전체의 결과입니다. 이 사례에서는 직접 수행한 구동·기구 작업을 중심으로 설명합니다.

SSAFY 공통 프로젝트 우수상 / 2026.08.10

THING · 시스템 구조

최종 발표자료의 원본 아키텍처 / 장치별 실행 환경과 통신 연결



팀 시스템의 연결

Jetson에서 손동작을 인식하고 Raspberry Pi의 ROS 2 제어 경로를 거쳐 로봇손을 구동합니다. 관제 화면은 영상과 상태를 받습니다.

직접 맡은 연결

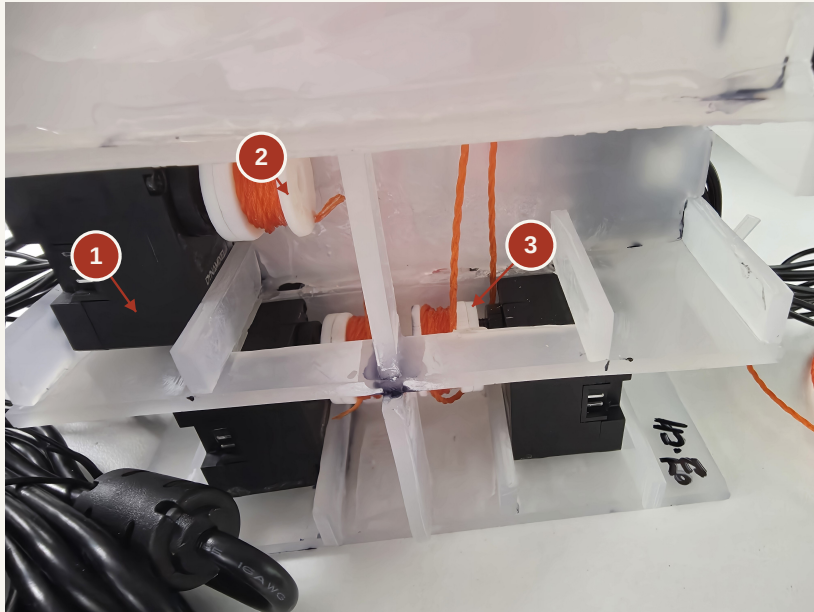
모터 통신 환경, 점검·제어 스크립트, 아크릴 고정부와 스푼·텐던 조립을 맡았습니다. 인식·관제·안전 로직은 팀의 협업 결과입니다.

기록 경로

Cyclone DDS는 내부 ROS 2 통신에 사용합니다. Jetson에서 EC2로 보내는 기록은 HTTPS 업로드입니다.

THING · 모터 제어와 기구 조립

모터가 스푼을 돌리면 텐던이 감기고, 연결된 손가락이 굽혀집니다.



구동부 내부: 모터·스풀·텐던 연결



전완부 모터 고정부

- 1 모터**
명령을 받아 회전을 만듭니다.
- 2 스푼**
모터와 함께 돌며 텐던을 감습니다.
- 3 텐던**
주황색 줄이 손가락을 당깁니다.
- 4 아크릴 고정부**
모터의 위치를 고정하는 틀입니다.

모터 통신 점검

7개 모터 ID 응답을 확인한 뒤 개별 모터를 움직였습니다. 연결 문제와 조립 문제를 구분해 확인하는 순서를 만들었습니다.

제어 스크립트

단일·키보드 제어, 원위치 복귀와 정지 스크립트를 작성했습니다. 조립 중 필요한 동작을 나눠 실행하도록 했습니다.

구동부 조립

아크릴 고정부를 제작하고 스푼·텐던을 통합했습니다. 감김 방향, 장력과 프레임 간섭을 실제 조립 상태에서 점검했습니다.

THING · 시연과 수상



엄지와 손가락으로 원통형 물체를 감싸 쥐는 동작

수상

SSAFY 공통 프로젝트 우수상

2026.08.10 / THING

[상장 보기](#)

시연한 동작

사람의 손동작 모방, 손가락 순차 동작, 캔과 부드러운 물체 파지를 시연했습니다. 모터 명령이 기구를 거쳐 실제 손가락 동작으로 이어지는 것을 확인했습니다.

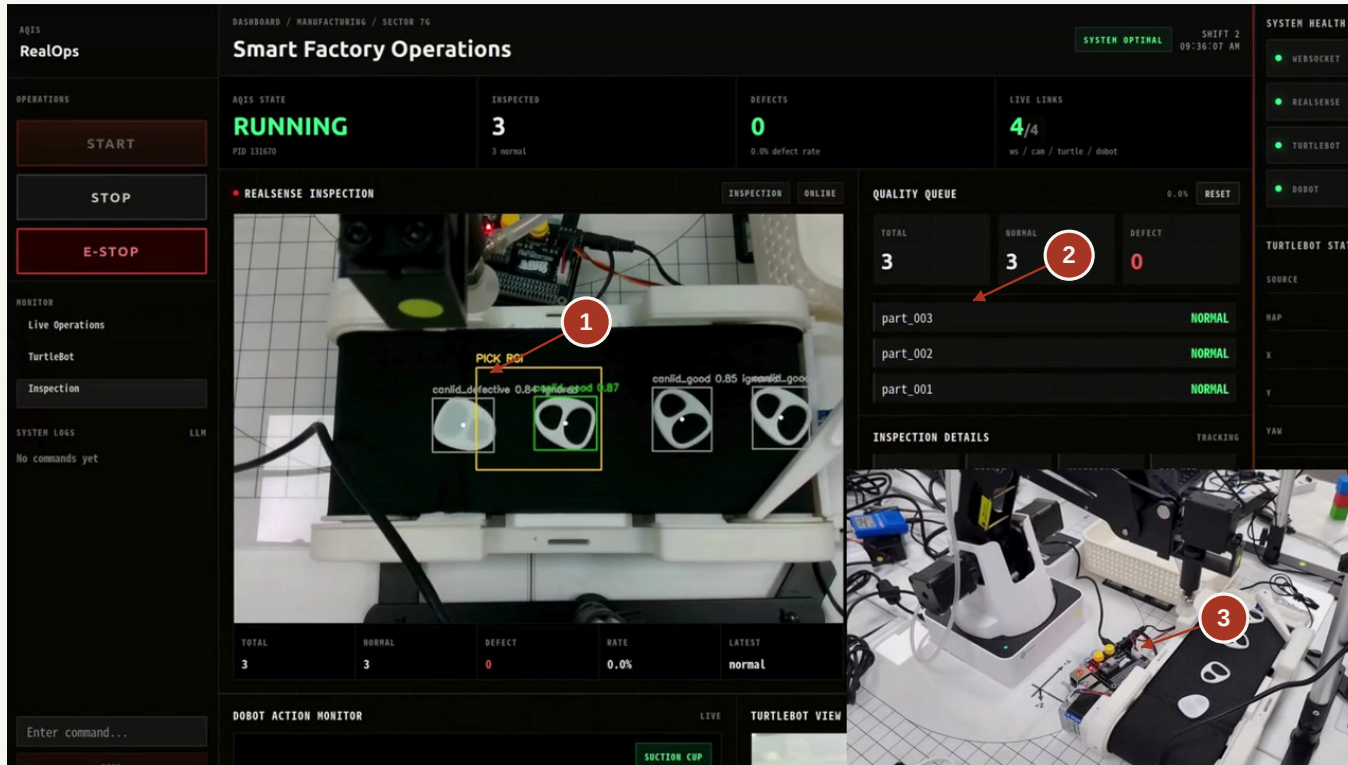
이 작업에서 배운 점

통신 성공과 원하는 동작의 완성은 다른 문제였습니다. 개별 모터 제어와 조립 상태를 나눠 확인하면서, 소프트웨어 명령과 물리적인 연결을 함께 다루는 경험을 했습니다.

다음 과제 반복 파지 성공률, 지연과 장력 변화를 같은 조건으로 측정해 동작의 재현성을 기록하는 것입니다.

AQIS · 스마트 팩토리

검사, 컨베이어, Dobot과 RealOps 관제를 연결한 스마트 팩토리 시스템



검출 영역, 작업 대기열과 실제 장비의 동작을 함께 보는 RealOps 관제 화면

직접 맡은 일 React 관제, FastAPI·WebSocket, ROS 2 연결, 장비 adapter와 Dobot 집기 시퀀스를 담당했습니다. LLM 명령과 키워드 fallback도 연결했습니다.

협업 팀원은 모델 학습·Roboflow·CAD·시뮬레이션을 맡았습니다.

2026.05 기획 / 06 본 개발

2인 팀 · 팀장

Full-stack & Robot Integration

1 검사 대상과 검출 영역

카메라에서 인식한 대상의 위치와 집기 영역을 확인합니다.

2 판정과 작업 대기열

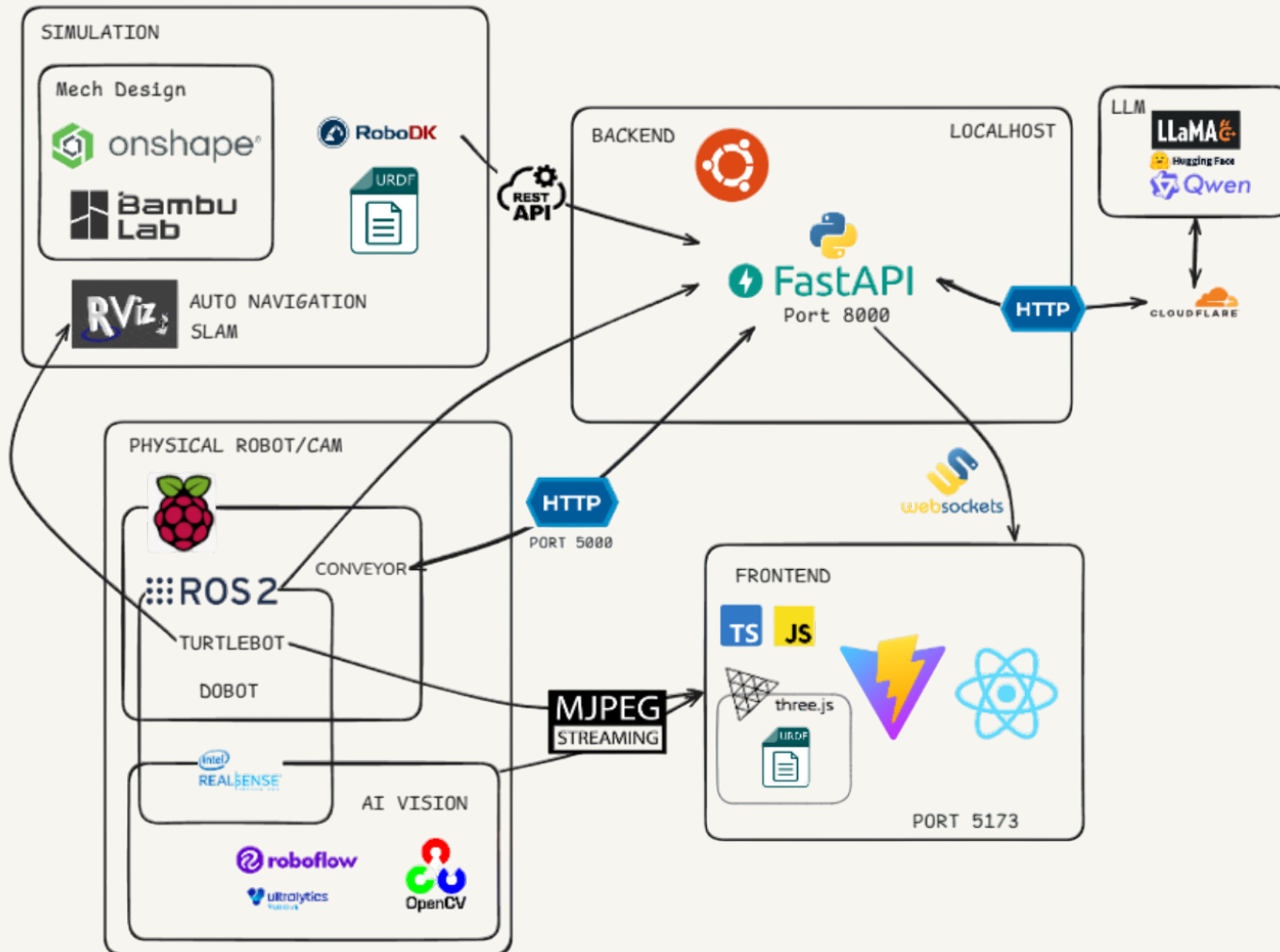
검사 결과와 처리할 작업을 관제 화면에서 확인합니다.

3 로봇·컨베이어 동작

물체를 옮기는 장비의 동작을 검사 화면과 함께 봅니다.

AQIS · 서버와 장비 연결

직접 그린 원본 아키텍처 / 시뮬레이션·장비·관제와 LLM 연결



공통 API

React 관제와 RoboDK 시뮬레이션은 FastAPI 서버에 연결됩니다. 서버가 검출·상태와 장비 명령을 다루고 WebSocket으로 관제에 전달합니다.

Mock으로 먼저 개발

가상 장비 응답으로 관제와 API를 먼저 개발한 뒤, 같은 인터페이스 아래 실제 장비 연결을 교체했습니다. 장비 adapter와 집기 시퀀스를 각각 점검합니다.

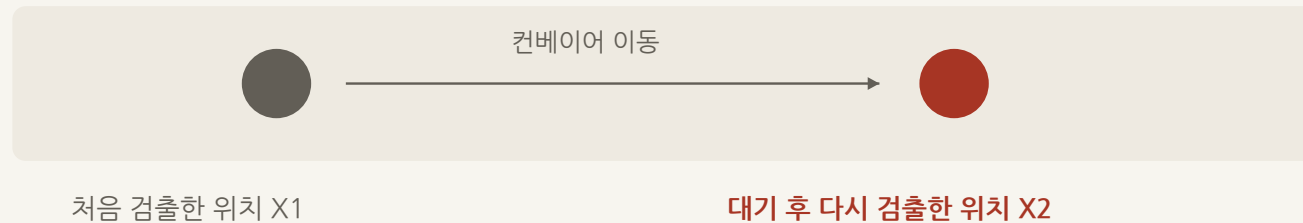
설계와 구현 범위

그림은 확장 설계를 포함합니다. TurtleBot은 SLAM·카메라·상태 관제까지 구현했으며 자동 임무는 후속 과제입니다. LLM 미연결 시 키워드 규칙을 사용합니다.

AQIS · 집기 좌표 갱신

문제: 컨베이어 위 대상이 이동하면, 로봇이 도착할 때 최초 검출 좌표는 이미 오래된 정보입니다.

검출 시점에 따른 위치 변화



좌표 재계산

X1을 계속 사용하지 않고, 정지 요청 후 준비 시점이 지난 검출에서 집기 좌표를 다시 계산합니다.

1

검출 수신

작업 조건 확인

2

> 정지 요청·대기

기본 대기 0.6초

3

> 새 검출 선택

시각·깊이 정보 확인

4

> 집기·분류·재개

성공·실패에 따라 분기

중복 집기 방지

객체 ID, 검출 영역 겹침과 중심 거리로 중복 여부를 판단합니다. 작업 중에는 집기 명령의 중복 실행도 막습니다.

현재 구현의 한계

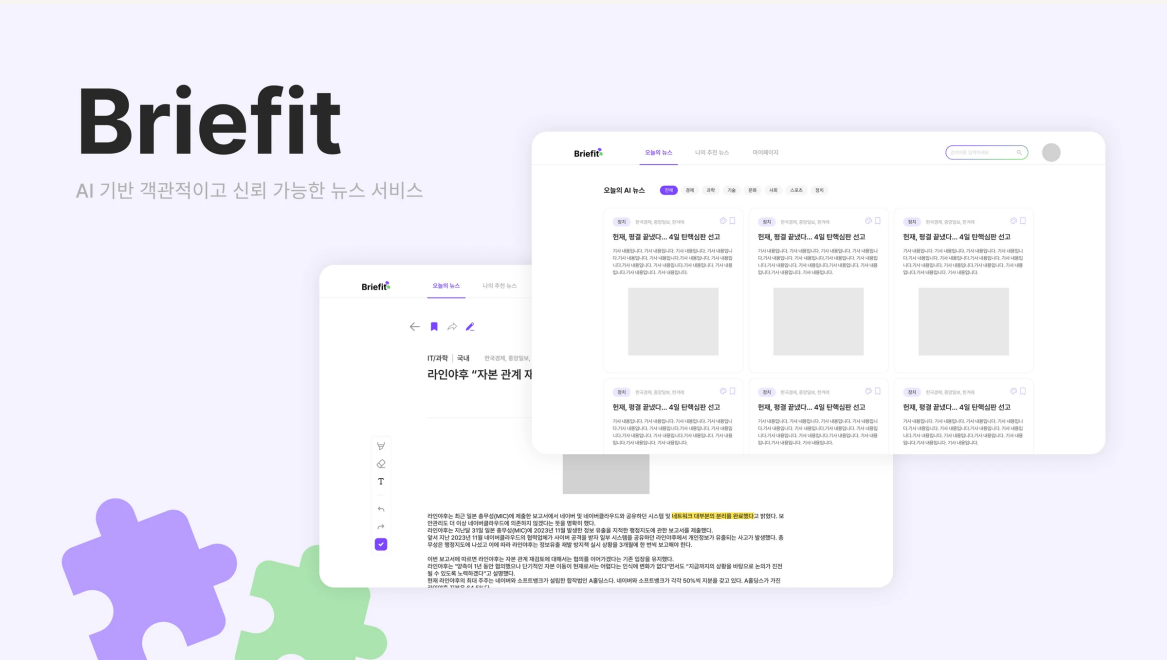
설정 시간 대기이며 센서로 정지를 확인하지는 않습니다. 시간 정보 없는 검출과 깊이 없는 고정 좌표 경로도 있어, 실물 적용 전 정지·좌표 검증을 보강해야 합니다.

Briefit · 뉴스 요약 서비스

기간
2025.05 - 09

팀·역할
6인 팀 / AI 2인
본인: AI 담당

사용 기술
Python / aiohttp
BeautifulSoup
Transformers / KoBART



뉴스를 모아 읽고 요약을 확인하는 서비스 / 제품 UI는 팀의 협업 결과입니다.

직접 구현한 범위

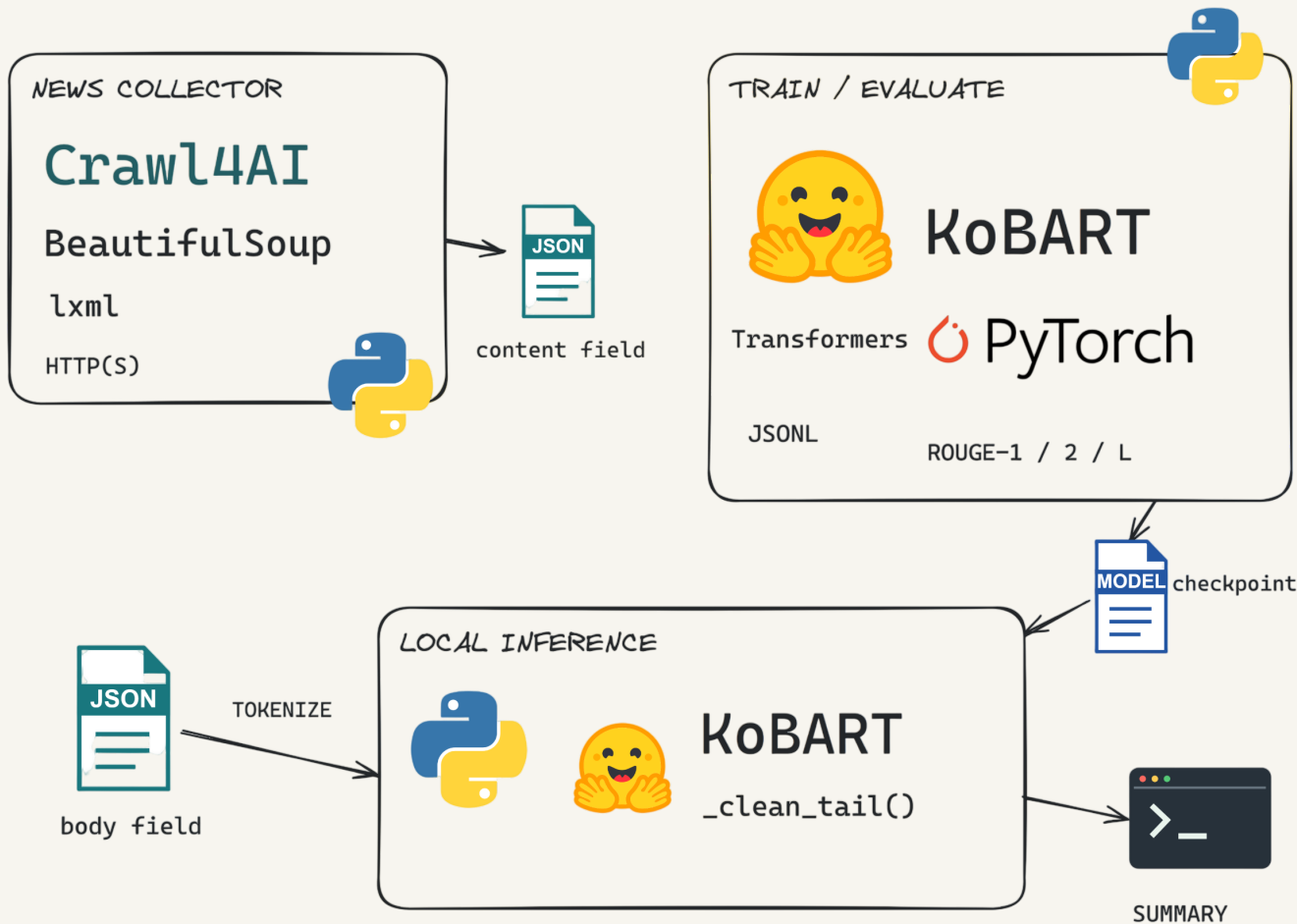
기사 수집 배치화, URL 필터·중복 제거,
KoBART 데이터 분할·학습·생성·ROUGE 평가
스크립트와 반복 종결문 후처리를 작성했습니다.

프로젝트 수상

IT대학 소프트웨어 공모전 금상
숭실 캡스톤디자인 경진대회 장려상
IT 프로젝트 프로리그 장려상
수상 날짜와 상장은 13쪽에 정리했습니다.

Briefit · 기사 수집과 요약 모델

2025년 KoBART 구현 / 수집·학습·추론·평가를 분리한 로컬 Python 파이프라인



학습 데이터 3,524건

학습 2,819건, 검증 352건, 평가 353건으로 약 8 : 1 : 1 분할했습니다. 평가에는 별도 기사와 기준 요약을 사용합니다.

수집과 요약 입력

URL 중복·덧글 URL·짧은 본문을 걸러 로컬 JSON으로 저장합니다. 수집 파일의 content와 추론 입력의 body는 필드 매핑이 필요합니다.

요약 후처리와 평가

생성 뒤 반복·짧은 끝문장을 정리합니다. ROUGE 평가는 후처리 전 생성문을 비교하므로 후처리 효과는 별도로 확인해야 합니다.

Briefit · 요약문 후처리와 수상

문자열 규칙을 적용하는 후처리 / 아래 문장은 동작 설명을 위한 예시입니다.

정리 전 · 예시

지역 도서관은 다음 주부터 주말 운영 시간을 늘린다. 이용자는 토요일 저녁에도 자료를 열람할 수 있다. **밝혔다. 밝혔다.**



반복되거나 짧은 끝 문장을 규칙으로 제거

정리 후

지역 도서관은 다음 주부터 주말 운영 시간을 늘린다. 이용자는 토요일 저녁에도 자료를 열람할 수 있다.

구현 선택 모델을 다시 학습하지 않고 생성 뒤 반복 표현을 정리하는 단계를 추가했습니다. 의미나 사실을 검증하는 기능은 아닙니다.

짧은 문장의 손실

«비가 온다.»처럼 유효하지만 짧은 문장도 빈 문자열이 됩니다. 반복 표현 감소와 함께 정보 누락을 비교해야 합니다.

Briefit 수상

2025 IT대학 소프트웨어 공모전 금상

2025.08.18 / Briefit

[상장 보기](#)

제15회 송실 캡스톤디자인 경진대회 장려상

2025.10.01 / Briefit

[상장 보기](#)

2025 IT 프로젝트 프로리그 장려상

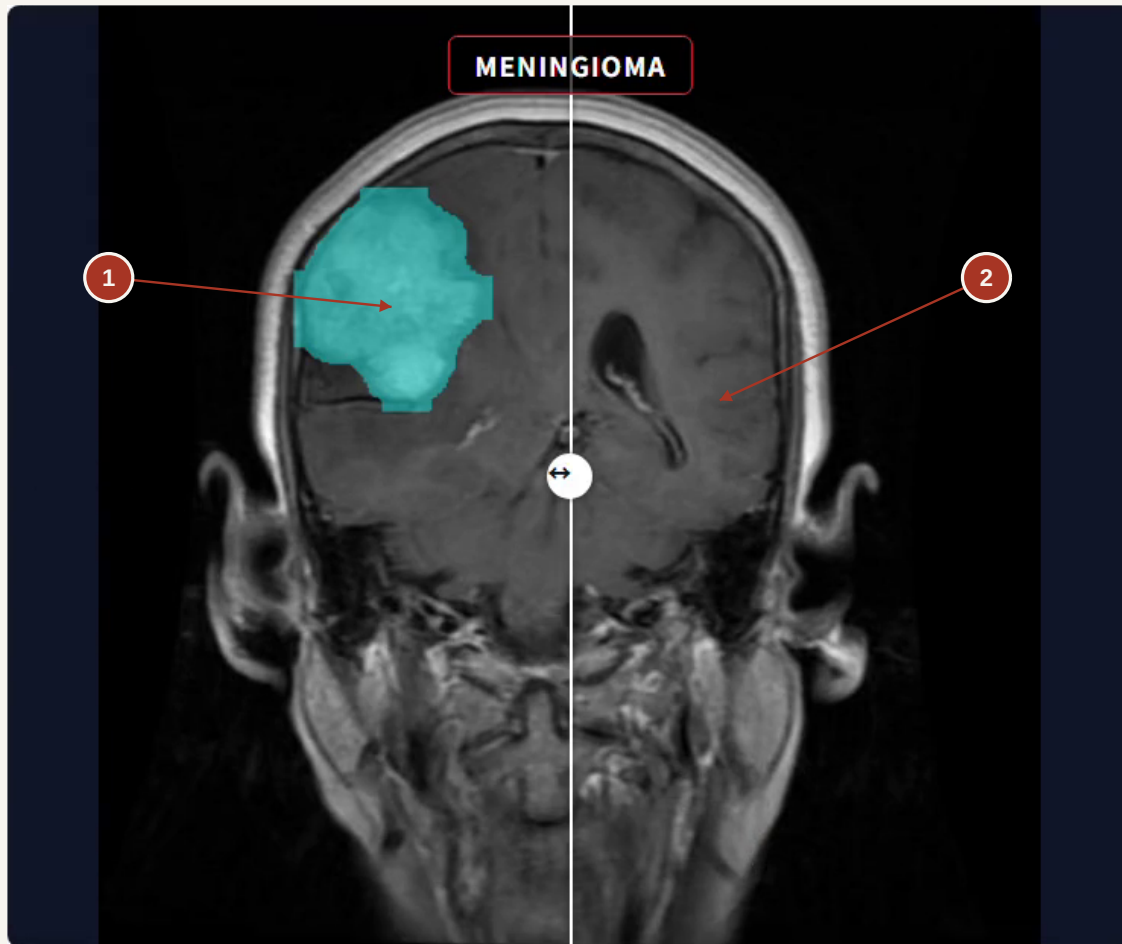
2025.11.22 / Briefit

[상장 보기](#)

상장 링크는 타인 수상자·학번·일련번호를 가린 전시용 이미지입니다.

Brain MRI · 종양 분류와 영역 분할

개인 프로젝트 / 2026.02 - 04 / Python · PyTorch · YOLO11 · OpenCV



① 왼쪽: 예측 영역 표시 ② 오른쪽: 원본
분류 결과는 수막종(Meningioma)으로 표시됩니다.

구현 범위

분류와 분할을 각각 수행하는 두 YOLO11 모델의 학습·추론 경로를 구성했습니다. 분류 범주와 분할 위치를 하나의 이미지에서 함께 볼 수 있게 했습니다.

입력 형식의 문제

원본 마스크를 그대로 쓰는 대신 분할 학습에 필요한 polygon label로 바꿔야 했습니다. 이진화, 형태학 처리, 외곽선 추출과 좌표 정규화를 연결했습니다.

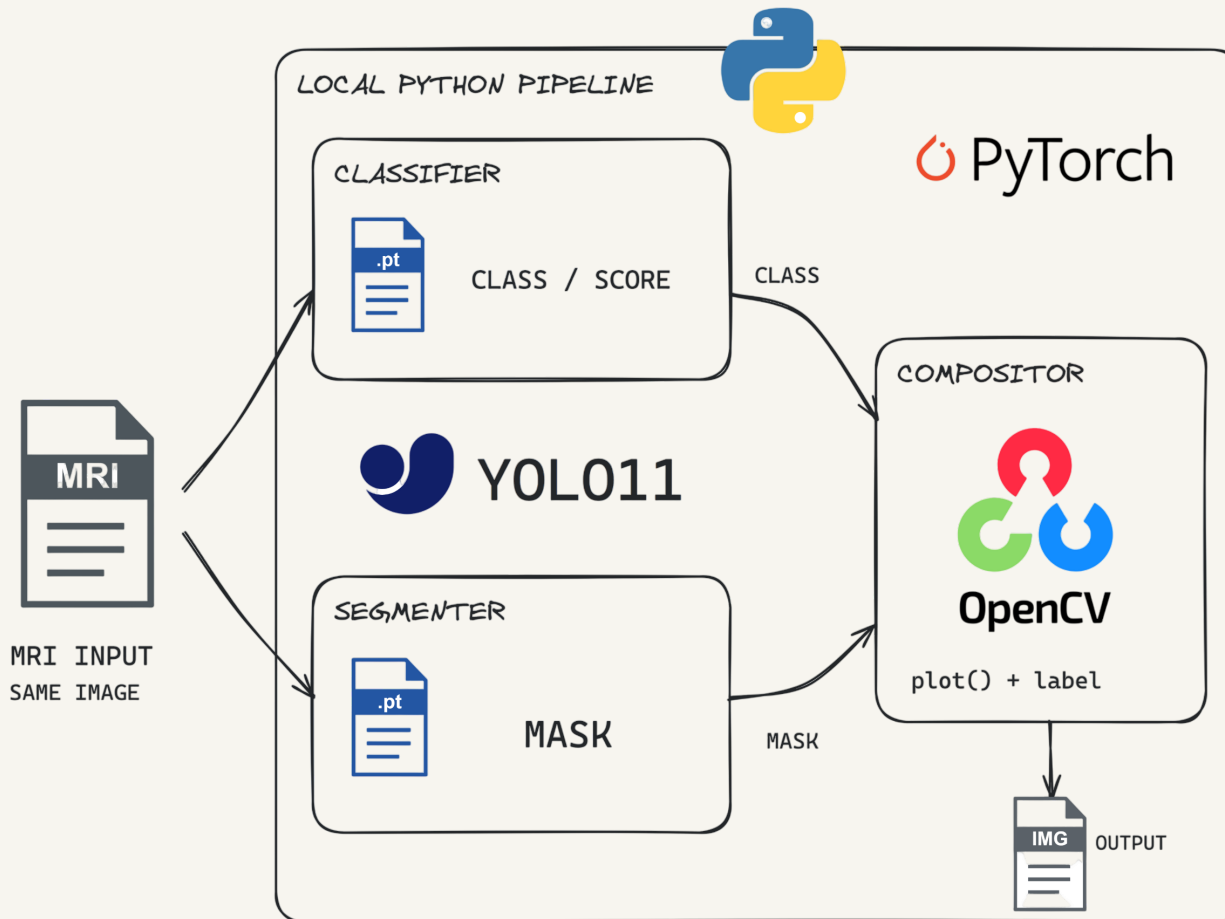
본인 범위

데이터 변환, 학습 설정과 통합 추론 코드를 작성했습니다. 두 모델을 독립적으로 학습하고, 결과를 한 화면에서 비교하도록 구성했습니다.

사용 목적 연구·학습용 프로젝트이며
임상 진단을 위한 검증은 수행하지 않았습니다.

Brain MRI · 분류·분할 모델 구조

같은 MRI를 두 모델에 각각 입력합니다. 한 모델의 결과를 다른 모델의 입력으로 사용하지 않습니다.



학습 데이터 준비

영역을 채운 마스크에서 테두리를 추출해 분할 모델의 학습 좌표로 바꿉니다. 다음 쪽에서는 이 변환 과정을 예시로 설명합니다.

현재 평가 조건

학습 코드는 test 경로를 검증용 val에 연결합니다. 최종 성능을 판단하려면 학습·검증에서 사용하지 않은 별도 데이터로 평가해야 합니다.

데이터 해석

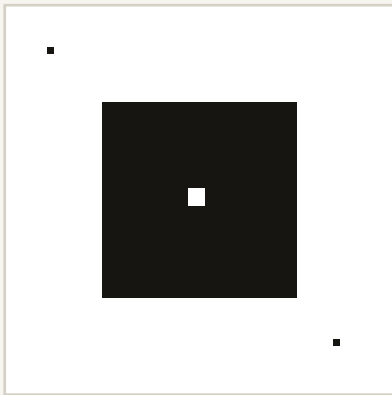
BRISC는 환자 식별 정보가 없어 환자 단위 독립성을 보장하기 어렵습니다. 비종양에도 병변이 포함될 수 있습니다. 임상 진단 검증은 수행하지 않았습니다.

학습 입력은 작업에 맞게 나눕니다. 분류는 폴더별 이미지, 분할은 마스크를 변환한 polygon label을 사용합니다.

Brain MRI · 마스크 전처리

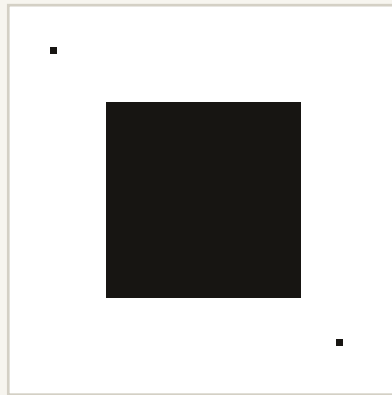
변환 과정 설명용 예시 / 검정은 영역, 흰색은 배경이며 실제 MRI나 모델 예측은 아닙니다.

1 예시 입력



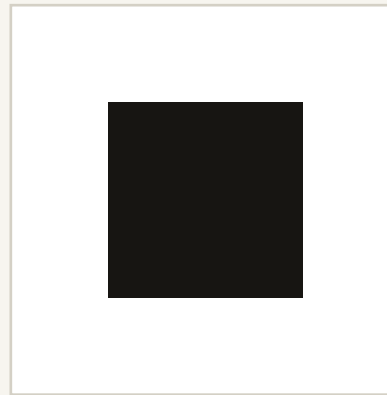
영역 안의 작은 구멍과
바깥의 잡음이 남아 있습니다.

2 구멍 메우기



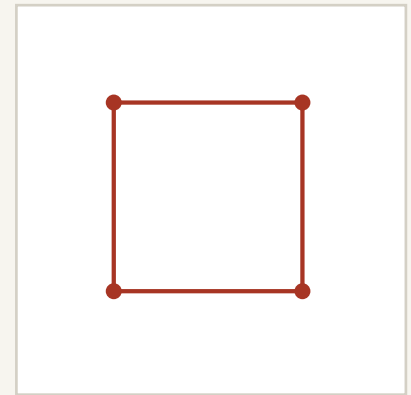
영역 안에 비어 있던
작은 구멍을 메웁니다.

3 작은 잡음 제거



영역 바깥에 떨어진
작은 점을 제거합니다.

4 테두리 좌표로 저장



테두리의 네 꼭짓점을
정규화한 좌표로 저장합니다.

마스크 변환

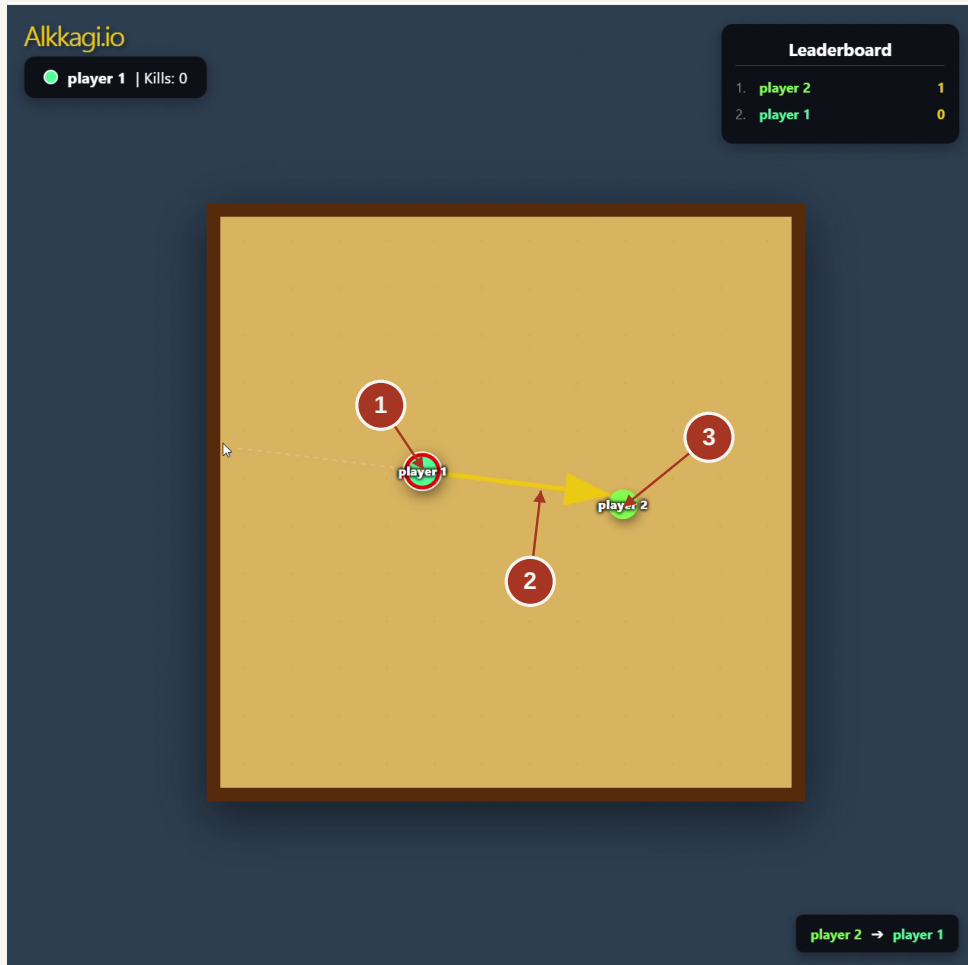
원본은 영역 내부를 채운 마스크 이미지입니다. YOLO 분할 학습에는 테두리 좌표가 필요하므로, 작은 구멍과 잡음을 정리한 뒤 외곽선을 추출했습니다.

학습 라벨 형식

종류를 나타내는 클래스 번호와 테두리의 꼭짓점을 저장합니다. 좌표를 이미지 너비·높이로 나눠 0~1 범위로 바꾸면 이미지 크기와 무관하게 위치를 표현할 수 있습니다.

Alkkagi.io · 실시간 알까기

개인 프로젝트 / 2026.03 - 04 / React · TypeScript · Node.js · Socket.IO



드래그로 조준하고 놓으면 내 돌이 발사됩니다.

상대 돌을 보드 밖으로 밀어내면
점수가 오르고 내 돌이 성장합니다.

1 내 돌

드래그를 시작하는 플레이어의 돌입니다.

2 방향·세기

노란 조준선으로 발사 방향과 세기를 표시합니다.

3 상대 돌

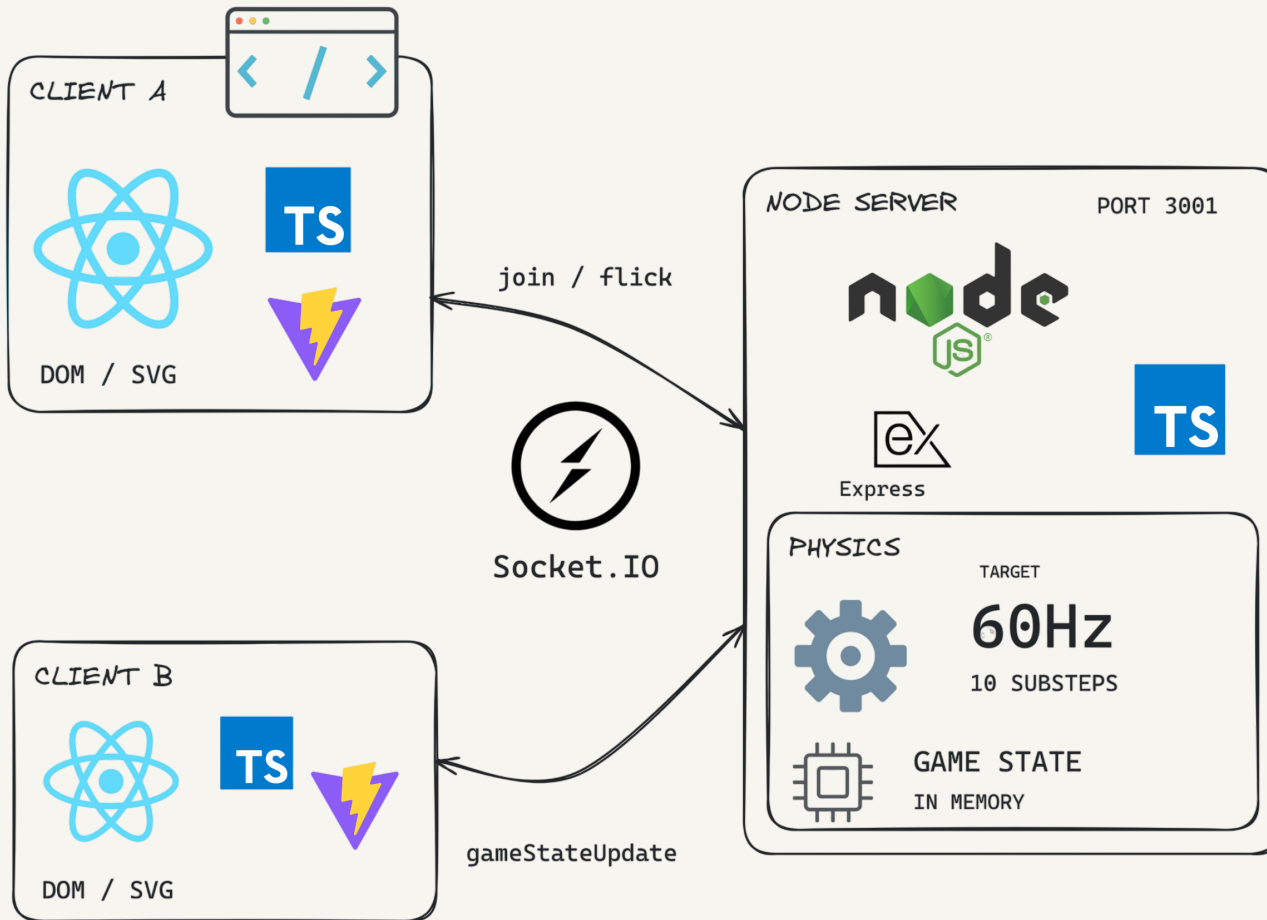
충돌로 상대를 보드 밖으로 밀어내는 것이 목표입니다.

직접 구현한 범위

조준 UI, 실시간 통신과 서버 물리를 구현했습니다. 서버가 위치·충돌·마찰을 계산하고 모든 플레이어에게 같은 상태를 전달합니다. 외부 물리 엔진 없이 TypeScript로 작성했습니다.

Alkkagi.io · 서버 물리와 상태 동기화

내 입력 → 서버의 충돌 계산 → 두 플레이어에게 같은 결과 / 상태는 서버 메모리에 유지합니다.



겹침 보정

충돌 순간 두 돌이 겹치면 겹친 거리의 절반씩 위치를 옮깁니다. 이미 서로 멀어지는 중이면 추가 충돌 힘을 적용하지 않습니다.

질량에 따른 충돌

반발계수, 상대 속도와 질량으로 속도를 갱신합니다. 점수가 오를수록 반경과 질량이 함께 증가해 돌의 충돌 특성이 달라집니다.

입력 제한과 마찰

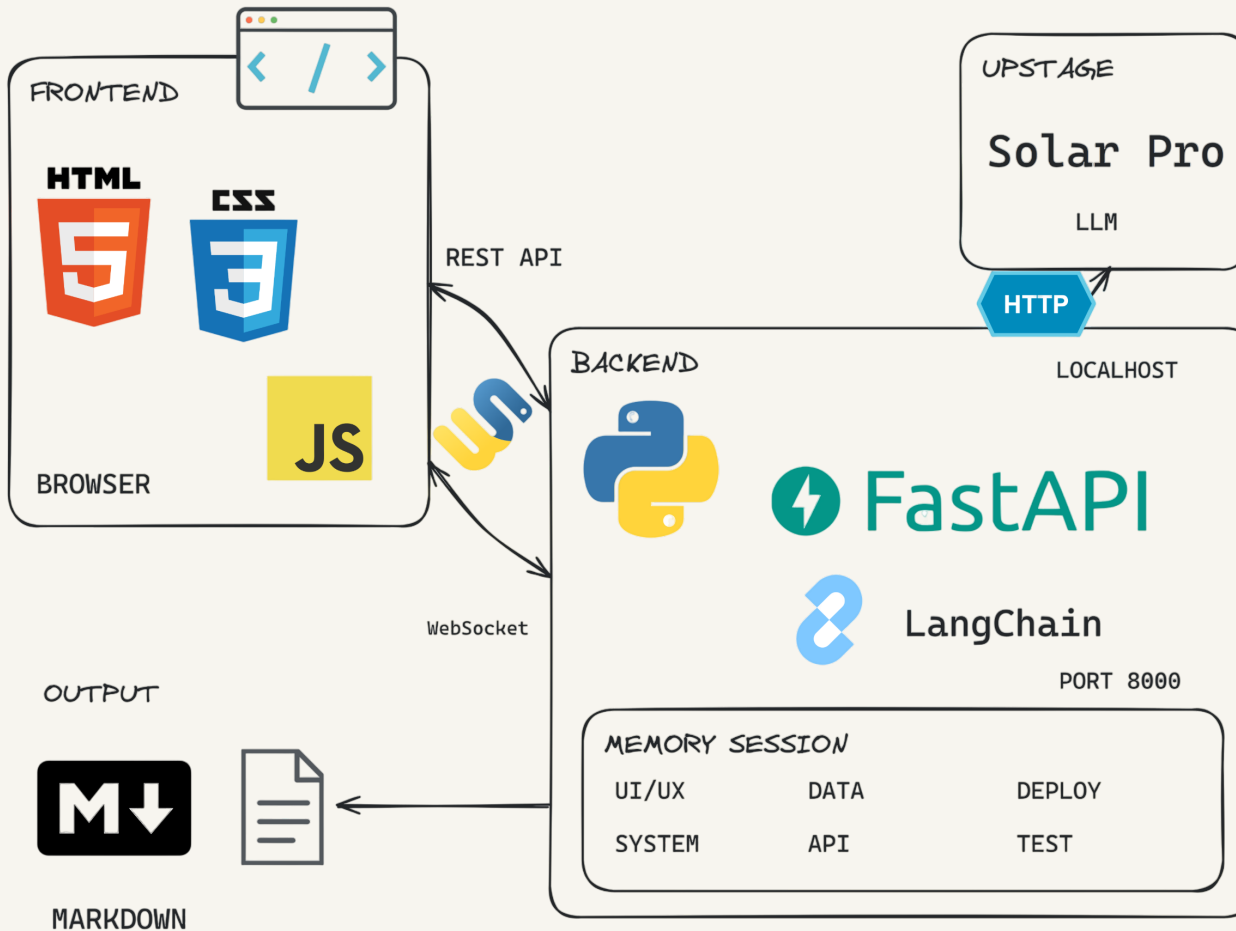
입력 간격을 500ms로 제한하고 발사 속도에 상한을 둡니다. 이동하는 동안 마찰을 적용해 속도가 점차 줄어들도록 했습니다.

서버가 유일한 기준 상태를 가지므로, 브라우저마다 다른 충돌 결과를 확정하지 않습니다.

후속 검증 동시접속자가 늘어날 때의 지연과 지속 프레임률은 별도로 측정해야 합니다. 서버 재시작 시 상태 복구도 추가 과제입니다.

Prompt Generator · 영역별 설계 대화

개인 프로젝트 / 2026 / FastAPI · WebSocket · LangChain · Solar Pro



영역마다 대화 이력·진행 상태·결과를 따로 유지합니다.

문제와 선택

화면, 데이터와 배포 질문이 뒤섞이면 필요한 조건을 빠뜨리기 쉽습니다. 기본 여섯 영역의 첫 질문을 병렬로 시작하고, 각 영역의 답변을 따로 누적하게 했습니다.

구현 구조

브라우저는 WebSocket으로 질문과 상태를 받습니다. FastAPI는 세션별 대화를 관리하고 LangChain·Solar Pro 호출로 질문과 프롬프트를 생성하도록 구성했습니다.

수정 흐름과 다음 과제

생성이 끝난 영역도 추가 대화로 수정할 수 있습니다. 세션은 메모리에만 있고 연결 종료 시 삭제되므로, 대화 저장과 재접속 복구가 다음 과제입니다.

연락처

Se Min Kong

공세민 · 소프트웨어 개발자

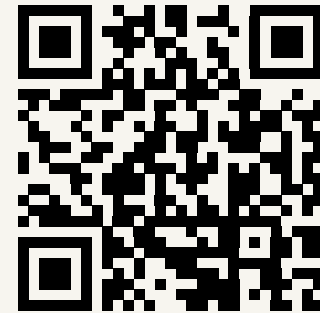
이메일

semin1224@gmail.com

GitHub

github.com/SeMinKong

웹 포트폴리오



seminkong.github.io/SeMinKong_Web/

프로젝트 영상과 상세 설명

이력서와 상장 전시